

The CUDA ClusterFinder

Kernel design · hardware limits · seven optimization steps

Khalil Daniel Ferjaoui

The kernel was never the bottleneck —^{*} feeding it was

One kernel, one thread per pixel, and seven steps to keep it fed

The stencil was fast almost immediately: at 3×3 the kernel needs 5.5 μs per frame while getting that frame across PCIe costs 16.6 μs , or 13.2 uncontended. The frame is gated by PCIe, not the arithmetic. Six of the seven steps get data in, get results back, and measure honestly.

WHERE THIS ENDS UP · THE SHIPPED f32 BUILD AFTER ALL SEVEN STEPS · ENGINE TIMES [f32 · s4] · RECONCILED IN A1

×9.1

VS 24-THREAD CPU

61,312

FRAMES / SECOND

16.3 μs

PER FRAME, END TO END

6 / 23 M

CLUSTER MISMATCH VS CPU TWIN

RTX 4090 (Ada, sm_89) · PCIe 4.0 ×16 · Mönch 400×400 uint16 · 3×3 clusters · 100 000 frames · Cu fluorescence, MAX IV

* This em dash was hand written (not AI generated ☺)

CONTEXT · WHAT THE CODE DOES

What the kernel does, and what limits it

No optimizations yet, only what the hardware has to do.

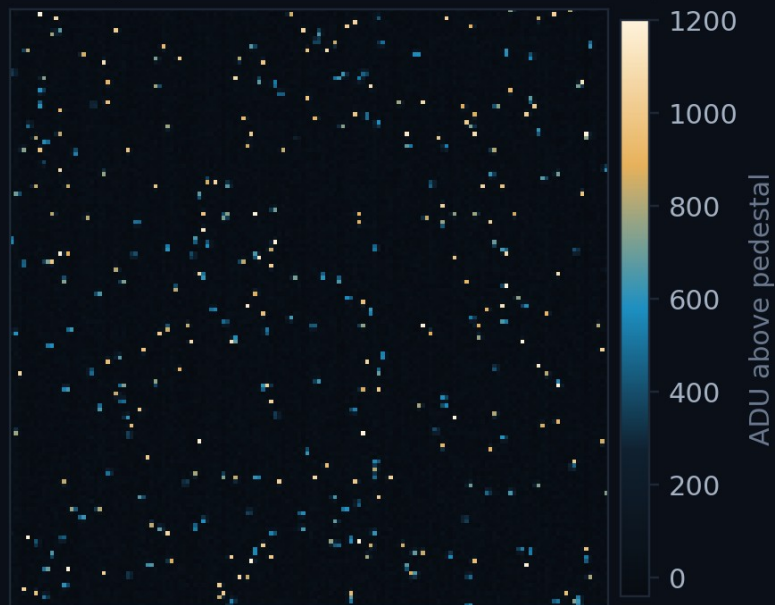
COMING UP

- 3–4 The algorithm
- 5–6 The two machines
- 7–8 The CUDA kernel
- 9–10 What limits it
- 11 The roadmap

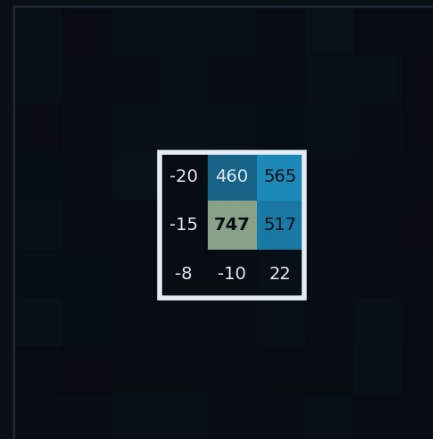
A photon is not a pixel — it is a cluster

- One photon's charge **spreads over neighbouring pixels**; summing the 3×3 patch recovers its energy.
- The histogram of those cluster energies **is** the measurement: peak and width give gain and **energy resolution**.

one frame, pedestal subtracted · 150×150 crop

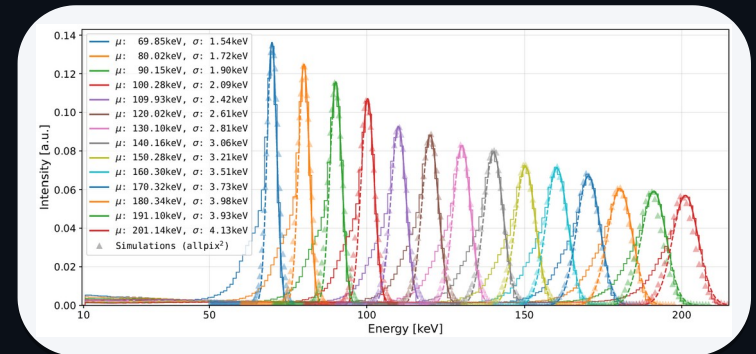


9×9 zoom on one hit



3×3 sum = 2258 ADU — one photon.
The peak pixel holds only part of the charge.

WHY IT MATTERS



Cluster-energy spectra from an energy scan.

MÖNCH03 · THE DETECTOR THIS FEEDS

ARRAY · PITCH · ACTIVE AREA

400 × 400 · 25 μm · 10 × 10 mm²

FRAMES PER SECOND

1.3 k standard, 3-6 k optimised

PEAK PIXELS = PHOTONS / FRAME

~2 330 · 1.5 %

Per pixel: subtract, threshold, update the pedestal

- Per pixel: subtract a **running pedestal**, keep what clears **$n\sigma \cdot rms$** , cut a 3×3 cluster around each local maximum.
- The pedestal is **updated by ~80 % of pixels every frame**, so arithmetic and data movement are coupled.

THE WHOLE ALGORITHM · THREE OUTCOMES, NOT TWO

```
// the whole algorithm, per pixel
v  = frame[i] - pedestal_mean[i]
rms = pedestal rms at i
m  = max(v) over the 3x3 window at i
if (m > nSigma * rms)           // a photon is within reach
    if (v == m) -> emit cluster   // ... and I am its peak
    else       -> nothing        // ... I am in its shadow
else                          // I saw nothing
    -> update pedestal
```

Thesis of this talk: the compute was fast almost immediately. Six of the seven steps are about feeding it.

THE SHAPE OF THE WORK

WORK ITEMS PER FRAME

160 000

OPERATIONS ON EACH

~5, identical

COMMUNICATION BETWEEN THEM

none

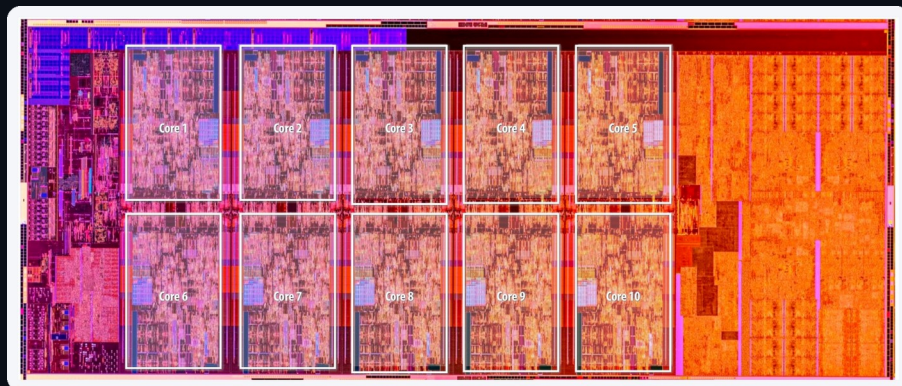
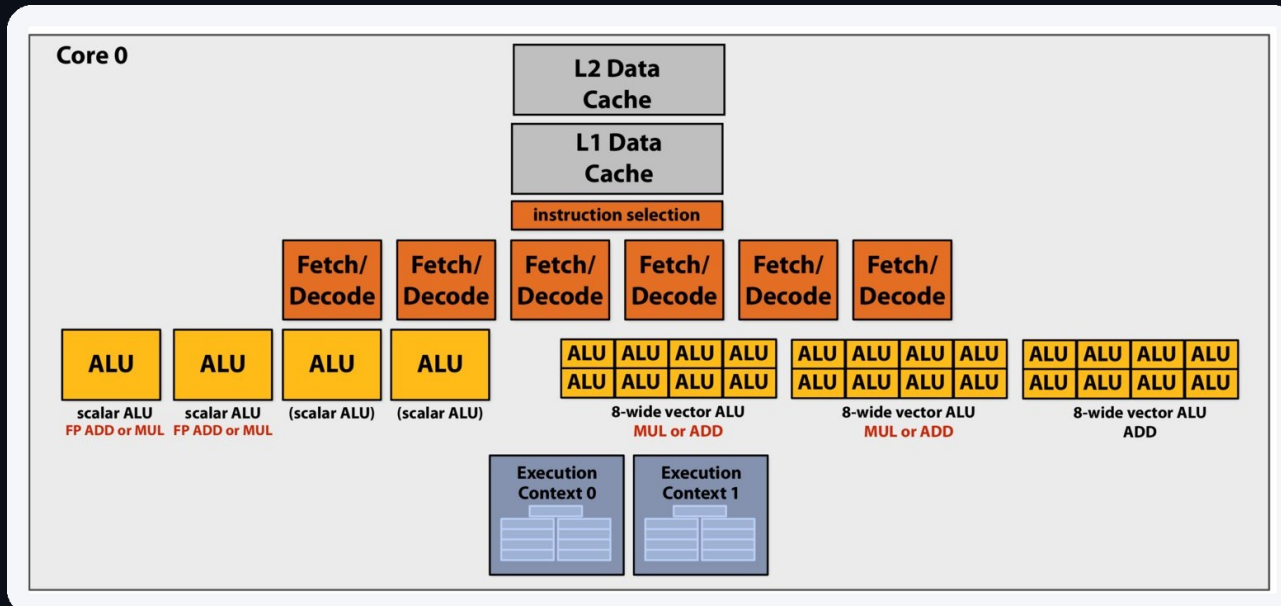
ORDER THEY MAY RUN IN

any, once the pedestal is fixed

BYTES PER FRAME

312.5 kB · ~2 330 clusters

CPU: latency-oriented, built to finish one thread fast



Count the boxes: **6**
fetch/decode and two levels
of private cache, all to keep
two instruction streams fed.
The ALUs are the small part.

Both diagrams after Stanford CS149, Fall 2025.

PC-MOENCH-04 · AMD RYZEN 9 7950X

CORES × SMT

16 × 2

CPU, 1 THREAD

1.75 ms

CPU MT, 24 THREADS

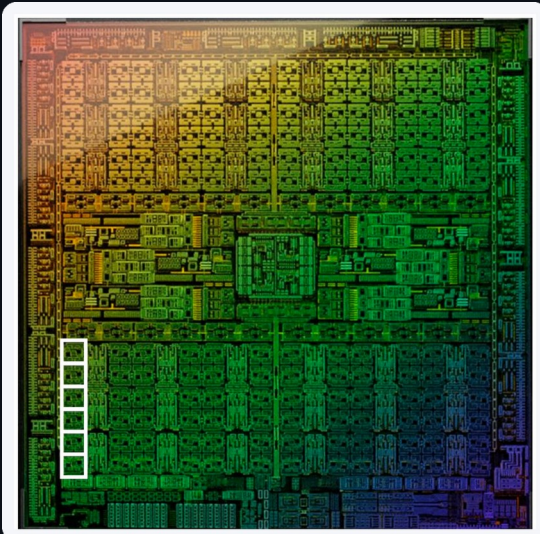
148 μs

THAT IS THE BAR

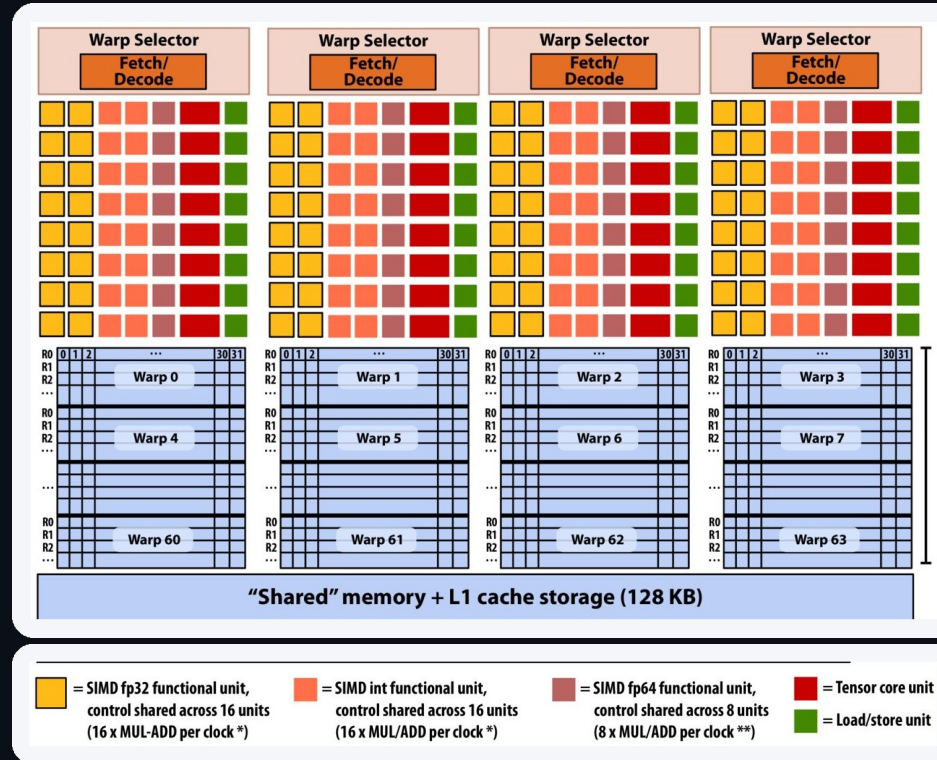
6 762 frames / s

Thread count is swept per cluster size, not assumed: 24 at 3×3, 32 at 9×9.

GPU: throughput-oriented, the whole frame at once



AD102 · 144 blocks, 128 enabled on this card. One SM boxed.



Same grammar, inverted proportions: **4 fetch/decode** for **64 warp contexts**.

One SM: a V100 is shown; this card's is the same idea: 128 FP32 lanes, 48 warp slots, 100 kB shared memory. After Stanford CS149, Fall 2025.

NVIDIA GEFORCE RTX 4090

FP32 LANES

16 384

128 SMS × 128 LANES

vs 32 CPU streams

RESIDENT THREAD SLOTS

196 608

THE FRAME NEEDS

160 000 → all at once

Device memory runs at 1 008 GB/s (384-bit, 21 Gbps); PCIe delivers 25. Two copy engines, so H2D and D2H run at the same time.

One thread per pixel

1 load tile + halo

>

2 __syncthreads

>

3 stencil reduction

>

4 classify

>

5 append or update pedestal

• A **16×16 block = 256 threads**; the grid tiles the frame in **625 identical blocks**.

• The output is **sparse**, but the **decision work is dense**: every pixel is tested, **160 000 times per frame**.

LAUNCH CONFIGURATION · ClusterFinderCUDA.hpp

```
block = dim3(BLOCK_X, BLOCK_Y);           // 16 x 16
grid  = dim3((ncols + BLOCK_X - 1)/BLOCK_X,
             (nrows + BLOCK_Y - 1)/BLOCK_Y);
device::find_clusters_in_single_frame<<<grid, block, shmem, stream>>>(
    d_frame, d_pd_mean, d_pd_sum, d_pd_sum2, d_pd_off, n_pd_samples,
    nSigma, nrows, ncols, d_clusters, d_cluster_count, max_clusters);
```

400×400 → a 25×25 grid of 16×16 blocks = 625 blocks per frame, over 128 SMs. Nothing in the launch depends on the cluster count.

THREE OUTCOMES, NOT TWO

```
// ClusterFinder.hpp · the serial CPU
if (max > nSigma * rms) {
    if (value < max)
        continue;
    // Not max go to the next pixel
    // but also no pedestal update
} else {
    pedestal.push_fast(iy, ix, ...);
}
```

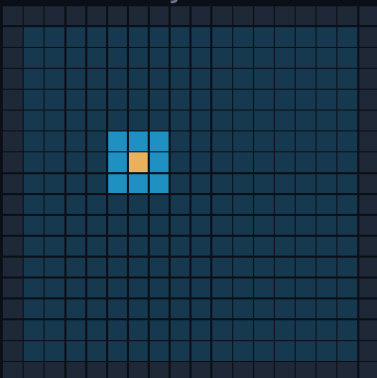
A shadow pixel is **neither recorded nor fed back**.

~80 % of threads update the pedestal, ~18 % shadow, ~1.5 % peaks.

Load the tile once, reuse it nine times

- Neighbouring threads need **overlapping** windows; without shared memory each pixel is fetched up to **nine times**.
- Each block stages one **pedestal-subtracted** tile plus its halo, loaded by the threads on the block edges.

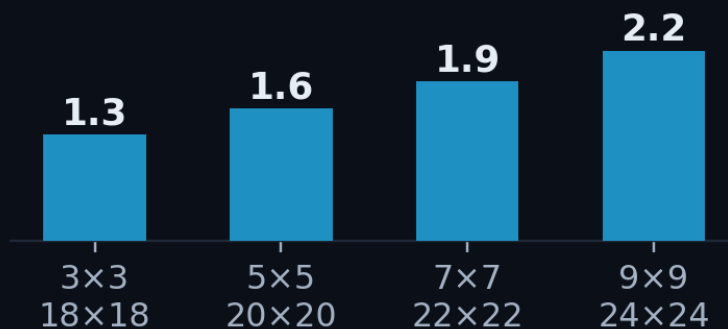
shared-memory tile · 18 × 18



- the thread's own pixel
- its 3×3 neighbourhood
- halo — loaded, never centred on

KB of shared memory per 16×16 block (float tile)

100 KB available per SM on Ada
— shared memory is never the limit



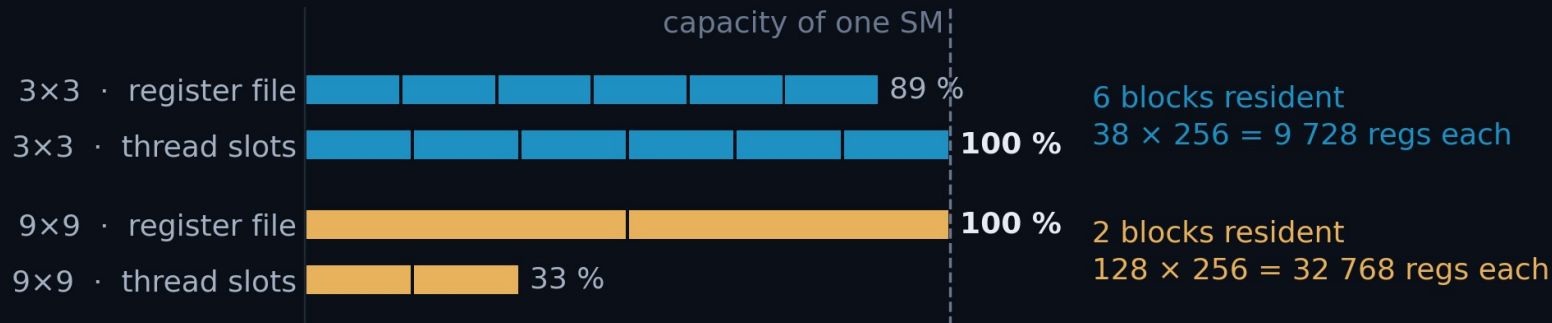
`clusterfinder_kernel.cuh`

```
extern __shared__ unsigned char smem[];
auto *sh = (COMPUTE_TYPE*)smem;
auto stride = blockDim.x + 2*col_radius;
auto tid = (threadIdx.y +
            row_radius)*stride
            + (threadIdx.x + col_radius);
// pedestal subtraction fused into the
load
```

Only **odd** cluster sizes are supported, so the centre pixel is unique.

38 registers per thread at 3×3, 128 at 9×9

- An SM has **65 536 registers** and **1 536 thread slots**; whichever runs out first decides how many blocks fit at once.
- Every thread keeps a private **clusterData[CSX × CSY]**, so register demand grows with the **square** of the cluster size. Neither build spills.



9×9: the register file is exactly full at two blocks ($2 \times 128 \times 256 = 65\,536$). **3×3:** the slots run out first, and the registers still have room.

MEASURED, NOT ESTIMATED · READ FROM THE BUILT .SO

```
cuobjdump -res-usage build/aare/_aare_cuda*.so | c++filt
3x3: REG:38  STACK:0  LOCAL:0      # STACK/LOCAL 0 = no spills
9x9: REG:128 STACK:0  LOCAL:0
```

PER SM · SM_89 · F32 BUILD

3×3 · BLOCKS RESIDENT

6

9×9 · BLOCKS RESIDENT

2

SPILLS, EITHER CASE

0 bytes

3×3 ON THE F64 BUILD

47 regs → 5 blocks

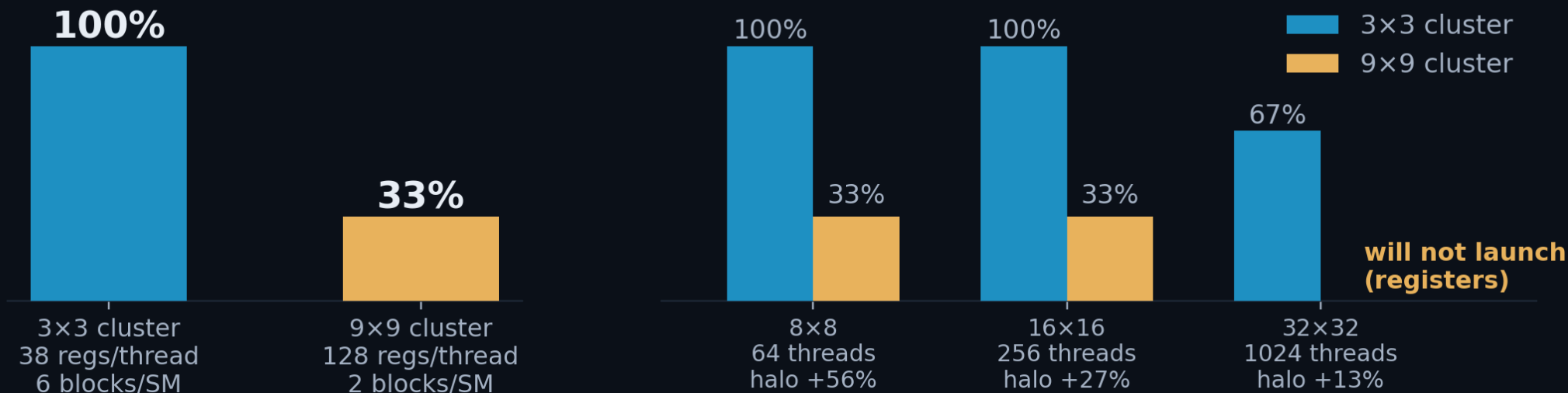
Occupancy is a latency-hiding budget

- When a warp stalls, the SM switches to another **already-resident** warp. **Occupancy = resident warps / the maximum** — how many alternatives it has to switch to.

BLOCK SIZE	THREADS / BLOCK	THREAD SLOTS / SM	MAX WARPS / SM	OCCUPANCY 3×3 · 9×9
16 × 16	256	1 536	48	100 % · 33 %

achieved occupancy, 16×16 block · f32 build

occupancy vs block size (halo overhead quoted for 3×3)



16×16 is the balance point. 33 % is not a failure to fix: it is what the register budget allows, and at 9×9 one kernel nearly fills the machine on its own.

Three acts, ordered by which bar is tallest

ACT I · FEED THE GPU

the host cannot submit work fast enough at 3×3 [f64]

opt1	First CUDA port 1 stream, one launch per frame	×2.34
opt2	Streams + batching 4 streams, 2 000-frame batches	×3.66
opt3	Pipeline rework sync barriers removed	×4.32
opt4	Pinned memory DMA-speed host transfers	×5.69

ACT II · GET THE RESULTS BACK

the host copy is now the tallest bar at 3×3 [f64]

opt5	Host↔GPU overlap chunked submit / collect	×7.45
opt6	Zero-copy collection read in place, never copy	×8.65

ACT III · THE KERNEL

the kernel is the tallest bar, at 9×9 at 9×9 [f32]

opt7	FP32 pedestal + variance rewrite the only kernel change in the deck	kernel −41%
------	---	-------------

Speedups are 3×3 against the best CPU configuration, 24 threads.

ACT I OF III · FEED THE GPU

The host cannot submit work fast enough

The kernel was fast almost immediately. This act is about the host, and it is told at 3×3, where PCIe is the floor.

STARTING FROM

6 762 FPS

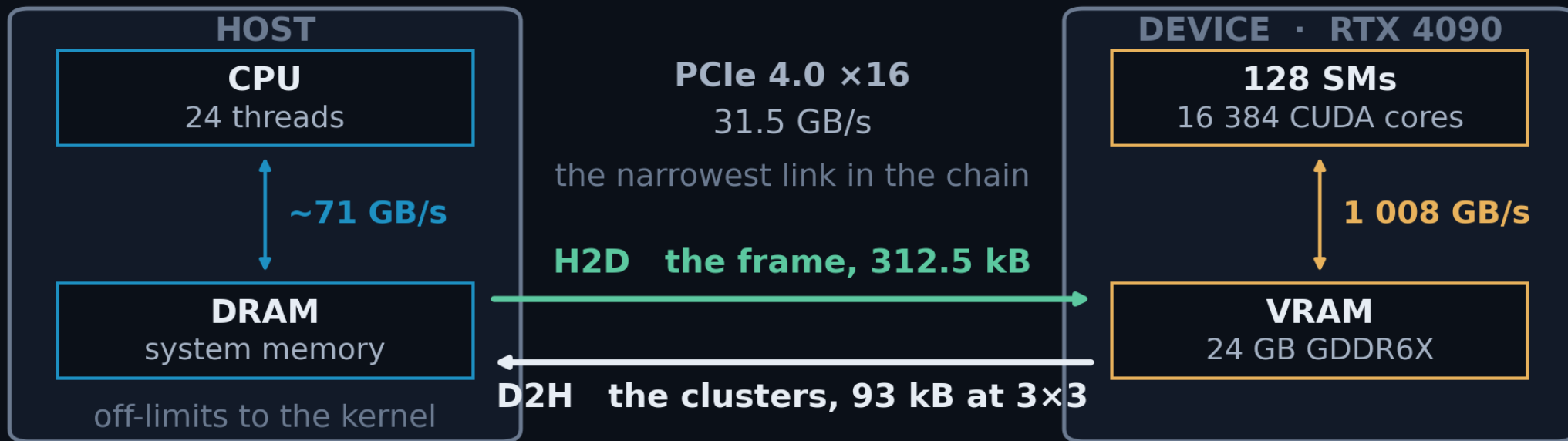
24-thread CPU · 14.8 s for 100 000 frames

COMING UP

- 12 two memories, one wire
- 13 opt1 · first port
- 14 opt2 · streams + batching
- 15 opt3 · no barriers
- 16 opt3 · one D2H, not two
- 17 opt4 · pinned memory

The GPU consumes 32x faster than PCIe delivers

- A CUDA kernel addresses **the GPU's own memory and nothing else**. So every frame is copied in — **H2D, host to device** — and every result copied back out — **D2H**. Those two names are what the next five slides are about.



The asymmetry is the whole talk. Host DRAM runs at ~71 GB/s and VRAM at 1 008, but everything between them crawls through **31.5**.

The first port runs at 26 % of the GPU's floor

- Shared-memory tiling with **halo loading** for any cluster size; pedestal subtraction fused into the tile load.
- One cudaMemcpy in, one kernel, one cudaMemcpy out; **the host blocks on every frame.**

```
ClusterFinderCUDAOpt2.hpp · find_clusters()
```

```
// one frame at a time, the host waits at every step
cudaMemcpy(d_frame, h_frame, bytes, cudaMemcpyHostToDevice);
find_clusters_in_single_frame<ClusterType, FRAME_TYPE>
    <<<grid, block, shmem>>>(d_frame, d_pd_mean, ...);
cudaMemcpy(h_out, d_out, out_bytes, cudaMemcpyDeviceToHost);
```

PCIe is full-duplex: H2D, kernel and D2H run on independent engines and overlap, so the **floor** is **max(H2D, kernel, D2H)**, never the sum. At 3×3 that is **16.2 μs → 61 859 FPS**.



OPT1 · 3×3 · 100 K FRAMES · F64

THROUGHPUT

15 807 FPS

VS 24-THREAD CPU

×2.34

PER FRAME

63.3 μs

H2D · KERNEL · D2H [S1]

13.2 · 5.5 · 6.4 μs

THE GPU FLOOR

61 859 FPS

% OF FLOOR

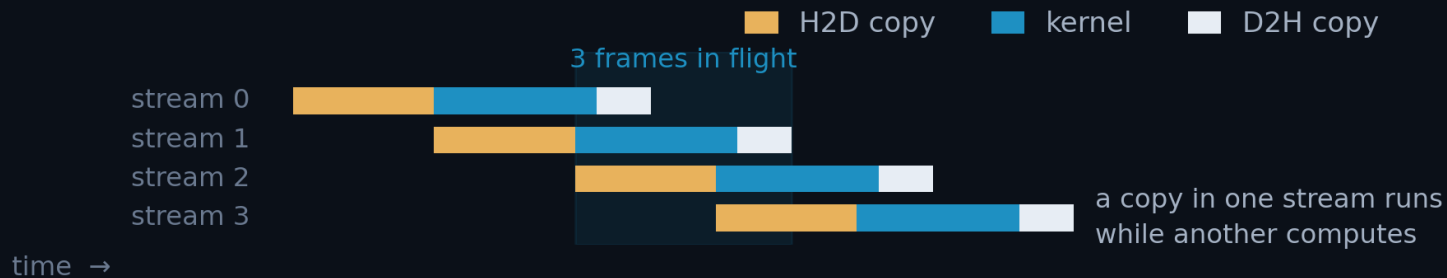
26 %

Four streams and 2 000-frame batches: ×1.56

- A **stream** is an ordered queue of GPU work. Work in **different** streams may overlap, so a copy can run while another stream computes.
- Each stream owns a **StreamContext** — frame buffer, output buffer, pedestal — and frames go **round-robin, 2 000 per call**.

ClusterFinderCUDA.hpp · per-stream state

```
struct StreamContext {
    cudaStream_t    stream;
    FRAME_TYPE      *d_frame;      ClusterType *d_clusters;
    PEDESTAL_TYPE   *d_pd_mean, *d_pd_sum, *d_pd_sum2;
};
auto &sc = v_sc[frame_idx % n_streams]; // round-robin
```



OPT2 · 3×3 · 4 STREAMS · BATCH 2 000

THROUGHPUT

24 726 FPS

VS 24-THREAD CPU

×3.66

PER FRAME

40.4 μs

STEP GAIN OVER OPT1

×1.56

% OF FLOOR · 61 859 FPS

40 % (was 26)

Scaffolding, not yet the payoff. The streams exist, but the host still synchronises after every round: see opt3.

One sync per batch, not one per round: ×1.18

opt1 · 1 stream, synchronous H2D copy kernel D2H copy



opt2 · 4 streams, sync barrier per round

barrier — H2D starves



opt3 · 4 streams, barriers removed



THE BARRIER, BEFORE AND AFTER

```
// opt2: barrier after every round
for (round) {
    submit(n_streams frames);
    cudaDeviceSynchronize();
}
// opt3: submit everything, sync once
for (frame : batch) {
    cudaMemcpyAsync(..., sc.stream);
    kernel<<<..., sc.stream>>>(...);
    cudaMemcpyAsync(..., sc.stream);
}
cudaStreamSynchronize(...); // once
```

29 188 FPS · ×4.32

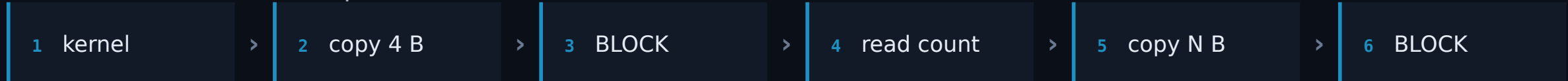
34.3 μs/frame · 47 % of floor (was 40)

- opt2 synchronised **all streams after every round**, draining the GPU empty each time.
- opt3 submits every frame's H2D → kernel → D2H **asynchronously** and synchronises **once per batch**.

Each lane is one stream. The panels are scheduled, not sketched: H2D and D2H are one FIFO engine each.

One D2H per frame, not two

- opt2 asked the device **how many clusters**, blocked until the answer came back, then asked for **that many**. The size of the second copy was a function of data that had not arrived yet.
- opt3 gives every frame a **fixed envelope** — count, then room for **cap** clusters — so the copy's size is known at construction and can be queued with the kernel. The count is still read, but **afterwards**, on the host.



You can only stream a transfer whose size is known in advance. The price is bytes: the envelope is sized by the **cap**, not by the clusters actually found. **3×3: 120 kB shipped for 93 kB of clusters. 9×9: 558 for 467.**

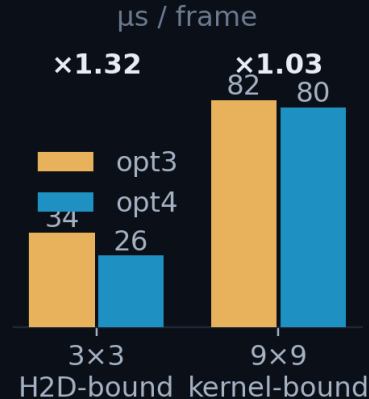
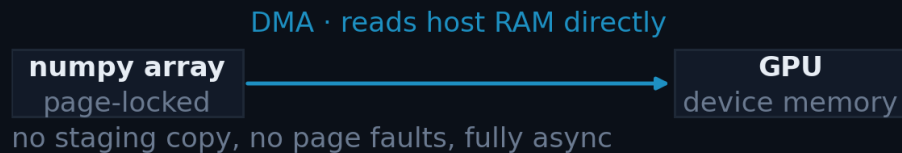
Pinning the input buys DMA-speed H2D: ×1.32

- Normal host memory is **pageable**, so the driver stages every transfer through a **hidden pinned buffer**: copied twice.
- Pinning** locks the pages in RAM, so the DMA engine reads host memory **directly**.

PAGEABLE · before opt4



PINNED · opt4



ClusterFinderCUDA.hpp

```
// pin the whole dataset once
cudaHostRegister(ptr, bytes,
                 cudaHostRegisterDefault);

// ... run the whole campaign ...

cudaHostUnregister(ptr);
```

38 486 FPS · ×5.69

26.0 μs/frame · 62 % of floor, the largest step in Act I

Measured H2D: one 312.5 KiB frame in **13.2 μs = 24.2 GB/s** — 77 % of PCIe 4.0 ×16, i.e. true DMA speed. **Pageable staging manages ~15 GB/s** for the same frame, because it is copied twice.

ACT II OF III · GET THE RESULTS BACK

The host copy is now the tallest bar

Both transfers are DMA already. What is slow is what happens AFTER the D2H: copying clusters out of the finder's pinned buffer into heap the caller owns. Host to host. Still 3×3, but 9×9 is where this act pays most.

ARRIVING AT

38 486 FPS

opt4 · 26.0 μ s per frame · 62 % of the GPU floor

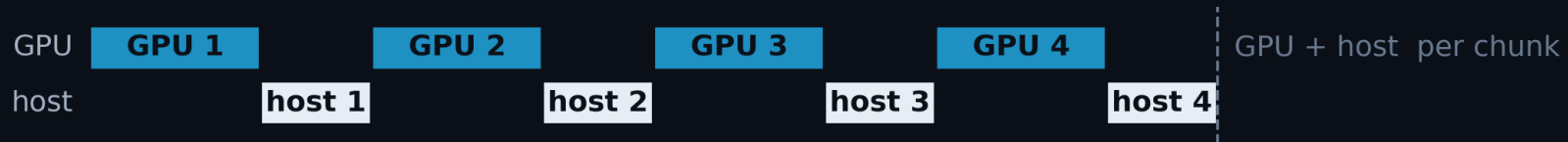
COMING UP

- 18 opt5 · host↔GPU overlap
- 19 9×9 · why overlap runs out
- 20 opt6 · zero-copy

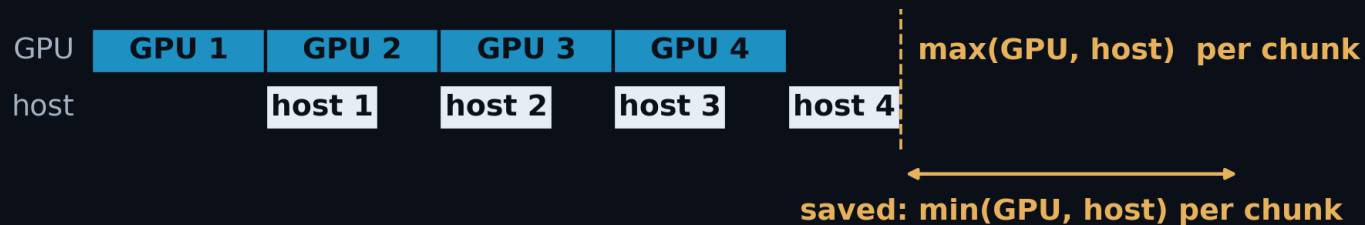
Overlapping host and GPU hides $\min(\text{host}, \text{GPU})$: $\times 1.31$

- opt3 overlapped the GPU's own engines, never the **host** with the GPU: the batch synchronised, then built ClusterVectors with the GPU idle.
- opt5 keeps **one batch in flight while materialising the previous one.**

submit → collect, serialized (opt4) · 3×3



submit(i+1) before collect(i) (opt5)



you never write these: find_clusters_batched() wraps them

```
tok = cf.submit_batch(data[a0:b0], first_frame=a0)
for a, b in bounds[1:]:
    nxt = cf.submit_batch(data[a:b], first_frame=a) // GPU starts i+1
    results.extend(cf.collect(tok)) // host unpacks i
    tok = nxt
```

OPT5 · 3×3 · NO CUDA WORK AT ALL

3×3 THROUGHPUT

50 410 FPS

PER FRAME · STEP · OF FLOOR

19.8 μ s · $\times 1.31$ · 81 %

THE WHOLE CHANGE

CUDA API CALLS ADDED

none

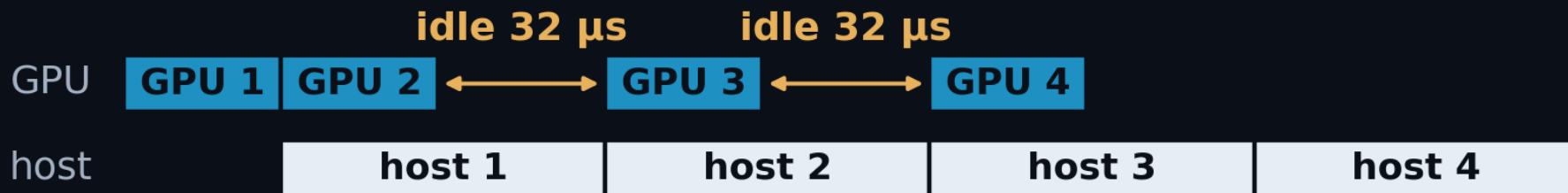
WHAT MOVED

the host loop

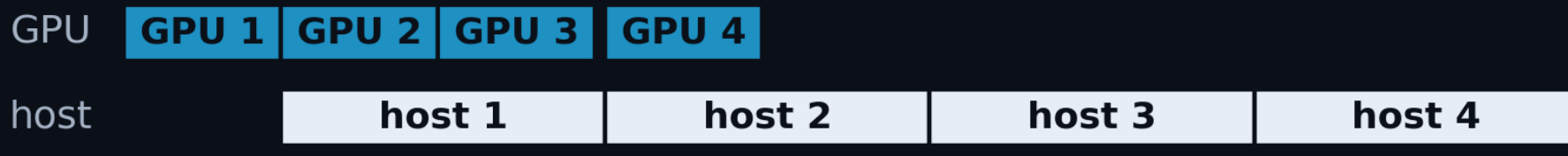
Overlap runs out: the host is the taller bar

- At 3×3 the host copy is **shorter than the GPU floor** and hides under it. At 9×9 it is **roughly twice the floor**, so overlap has less to hide: opt5 is worth **×1.20** here against ×1.31 at 3×3.

2 slots · what ships



3 slots · the natural next guess



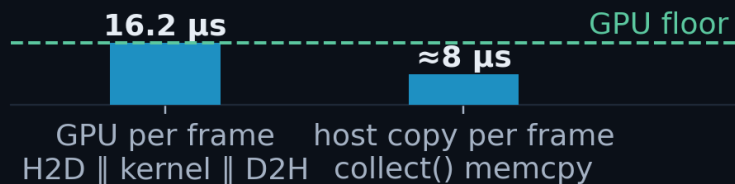
same finish
both ways

A deeper buffer **relocates the GPU's idle, it does not close it**: the host lane is already back-to-back in both strips, so it alone sets the pace. **The only way down is to make the host term smaller.**

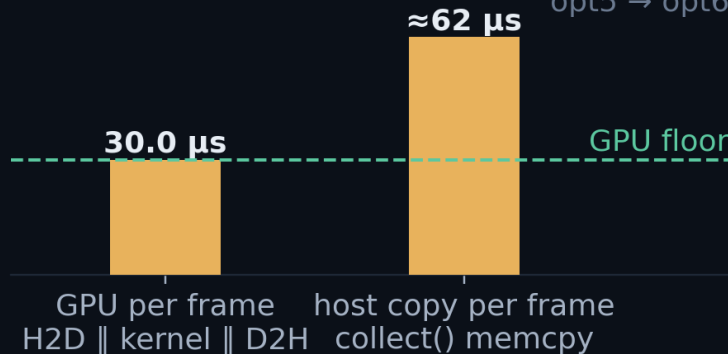
Read the results in place: $\times 2.21$ at 9×9

- The D2H lands in a **pinned host buffer**; collect() then allocates one ClusterVector per frame and memcpys into it — **467 kB per frame at 9×9** .
- collect_view() returns a **BatchView** onto that buffer instead: it withholds **ownership** past the chunk, not access.

3×3 · host copy 93 kB / frame
the copy hides under the GPU floor → small win
 $\times 1.16$
opt5 → opt6



9×9 · host copy 467 kB / frame
the copy is larger than the floor → cannot hide
 $\times 2.21$
opt5 → opt6



The win is **max(0, host copy – GPU floor)**. At 3×3 the copy already hid under the floor; at 9×9 it is **twice the floor** and cannot hide at any overlap.

OPT6 · 3×3 AND 9×9 · COLLECT_VIEW()

3×3 THROUGHPUT

58 495 FPS

PER FRAME · STEP · OF FLOOR

17.1 μ s · $\times 1.16$ · 95 %

9×9 THROUGHPUT

33 323 FPS

PER FRAME · STEP · OF FLOOR

30.0 μ s · $\times 2.21$ · 100 %

RESULTS

bit-identical to opt5

ACT III OF III · THE KERNEL

Only now is the kernel the tallest bar

The story moves to 9×9 , where the kernel is finally the tallest bar. First, how the engine times in this act are measured.

ARRIVING AT

58 495 FPS

opt6 · 3×3 , and 33 323 FPS at 9×9 , where this act pays

COMING UP

- 21 how the engine times are measured
- 22 opt7 · FP32 pedestal
- 23 catastrophic cancellation
- 24 why it comes last

Two configurations, one floor

s1

One stream, nothing else running

How long an operation actually takes.

s4

The shipped pipeline, four streams

How busy each engine is: a union, not a sum.

floor

Set by the busiest engine

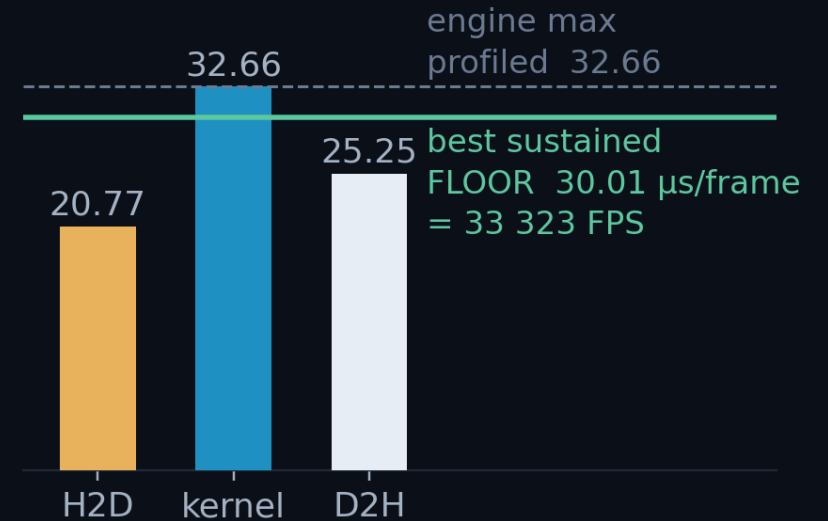
1 / the busiest engine, in $\mu\text{s}/\text{frame}$ or FPS.

s4 · the shipped pipeline, four streams · schematic, 9×9 shape



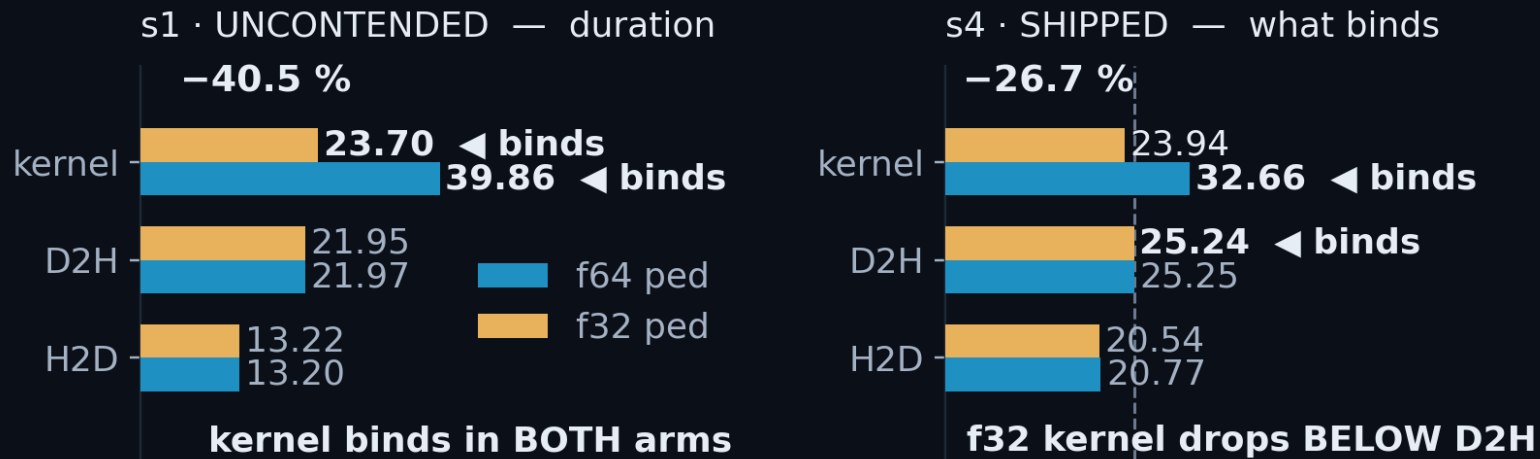
UNION — what s4 reports.
The kernels overlap, so it is shorter than their sum.

busy μs / frame · 9×9 f64 · measured



FP32 halves pedestal traffic: –41 % kernel time

- ~80 % of pixels take the **pedestal-update** branch: six accumulator values read and written.
- One typedef halves all four pedestal arrays — **48 bytes per updating pixel in FP64, 24 in FP32** — and the kernel is **bandwidth-bound**.



Two more effects, neither the reason it works: FP64 arithmetic runs at 1/64 of FP32 on a GeForce part, and the narrower accumulators free 9 registers at 3×3.

ONE TYPEDEF

```
// clusterfinder_kernel.cuh
using COMPUTE_TYPE    = float;
using DEVICE_PED_TYPE = float;
//                    was: double
```

Kernel, 9×9 [s1 · cap 1700]
39.86 → 23.70 μs (–40.5 %)

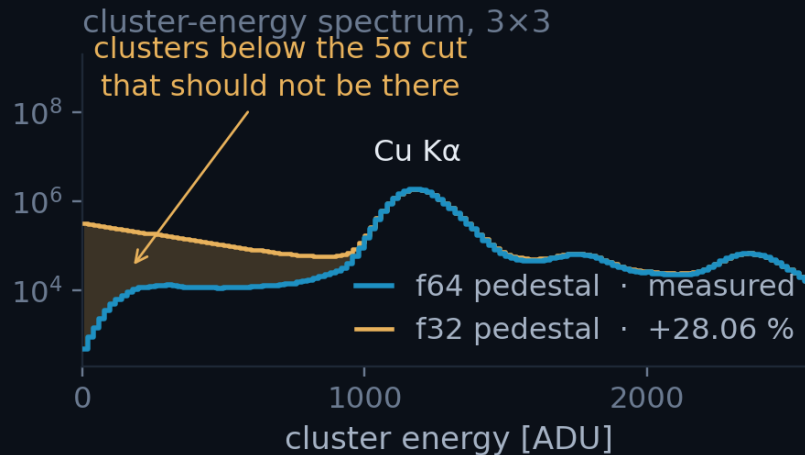
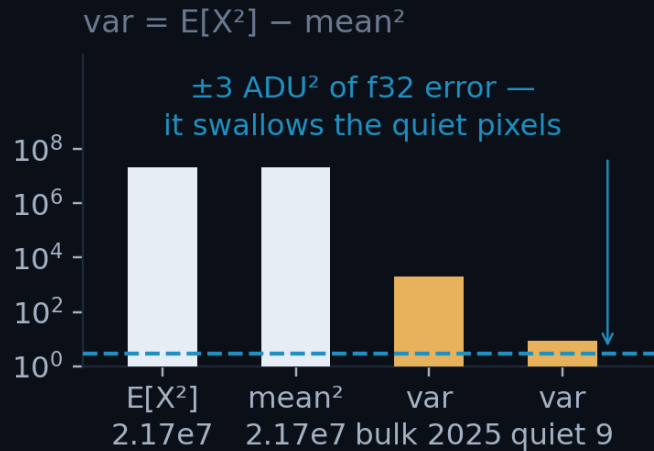
At 3×3 the same typedef is worth
–70.6 % in the kernel and **4.6 %** end
 to end: there the kernel was never the
 tallest bar.

Naive FP32 is **wrong** (next slide).

Both builds, same git rev, 20 000 frames. Full engine
 grid: A1.

Accumulate what is small, not what is large

- **The trap.** $\text{var} = E[X^2] - \text{mean}^2$ takes two numbers near 2×10^7 to get one near 2 000.
- **What it cost.** The rms **clamps to zero**, so a 5σ gate becomes a **0σ gate**.
- **The fix.** Accumulate the **centred** $Y = X - X_0$: both operands become $O(\text{rms})$.



Left: the operands and the answer against the $\pm 3 \text{ ADU}^2$ error floor. Right: the f32 curve is reconstructed — measured area, modelled shape. The two-line patch is in annex A5.

NAIVE F32 · WHAT IT DID

EXTRA CLUSTERS

+28.06 %

PIXELS AFFECTED

~1-2 % of the sensor

ERROR FLOOR

$\pm 3 \text{ ADU}^2$, absolute

AFTER THE REWRITE

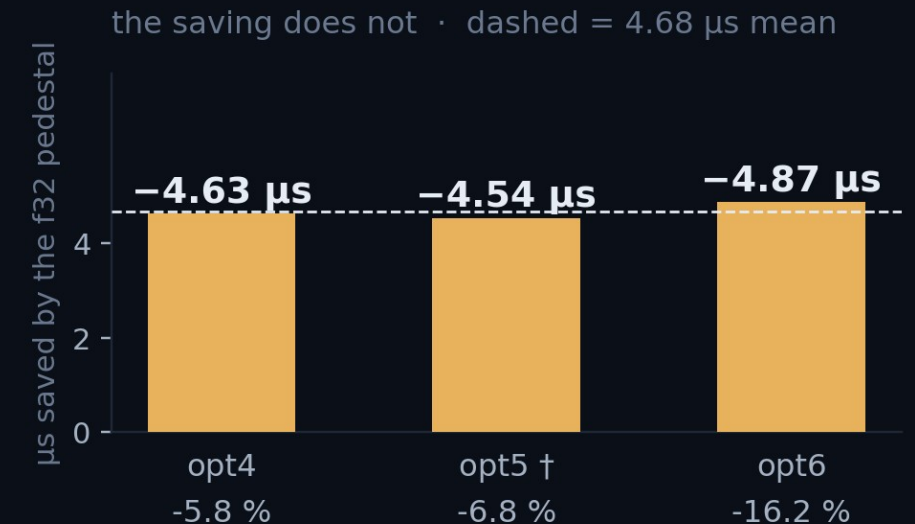
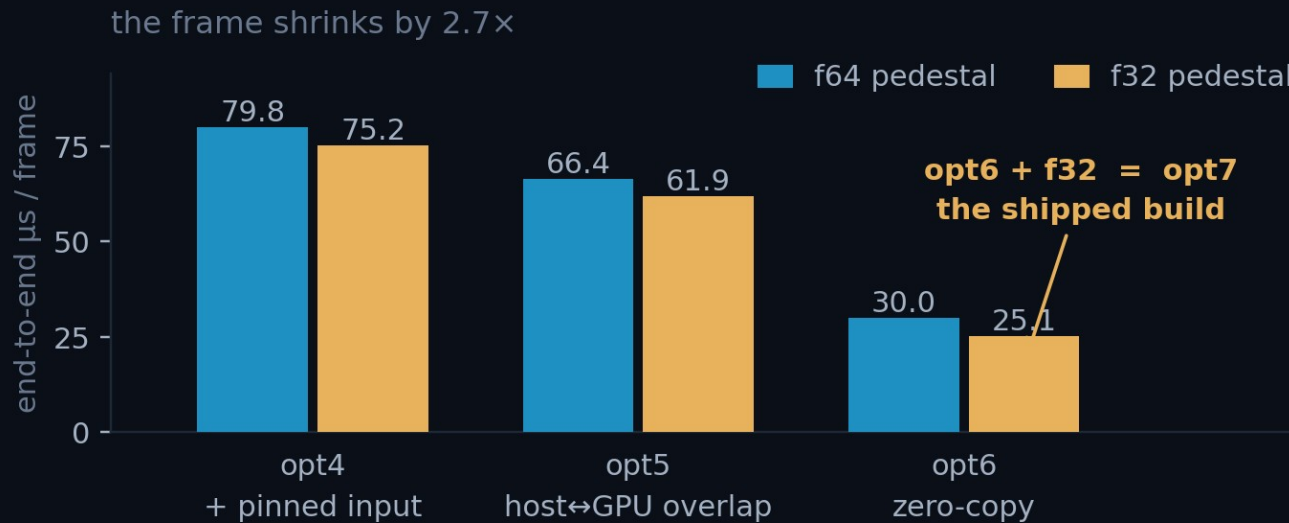
F32 VS F64 COUNTS

3×10^{-7}

VS THE CPU BASELINE

0.0039 %

The saving never grew — the frame around it shrank



The act ends by handing the floor away. At **s4** the f64 arm is kernel-bound, **32.66 μs** against a **25.25 μs** D2H. The typedef puts the kernel at **23.94**, below a D2H that never moved.

9×9 · 20 000 frames · 4 streams · cap 1 700 · warm · both arms at the same git rev. opt3 excluded and opt5 marked †: their arms sat in different allocator states. Worked through in annex A4.

RESULTS · WHAT CAME OUT OF IT

The whole ladder, and how to use it

Both cluster sizes end to end, and the audit behind the numbers.

ARRIVING AT

58 495 FPS

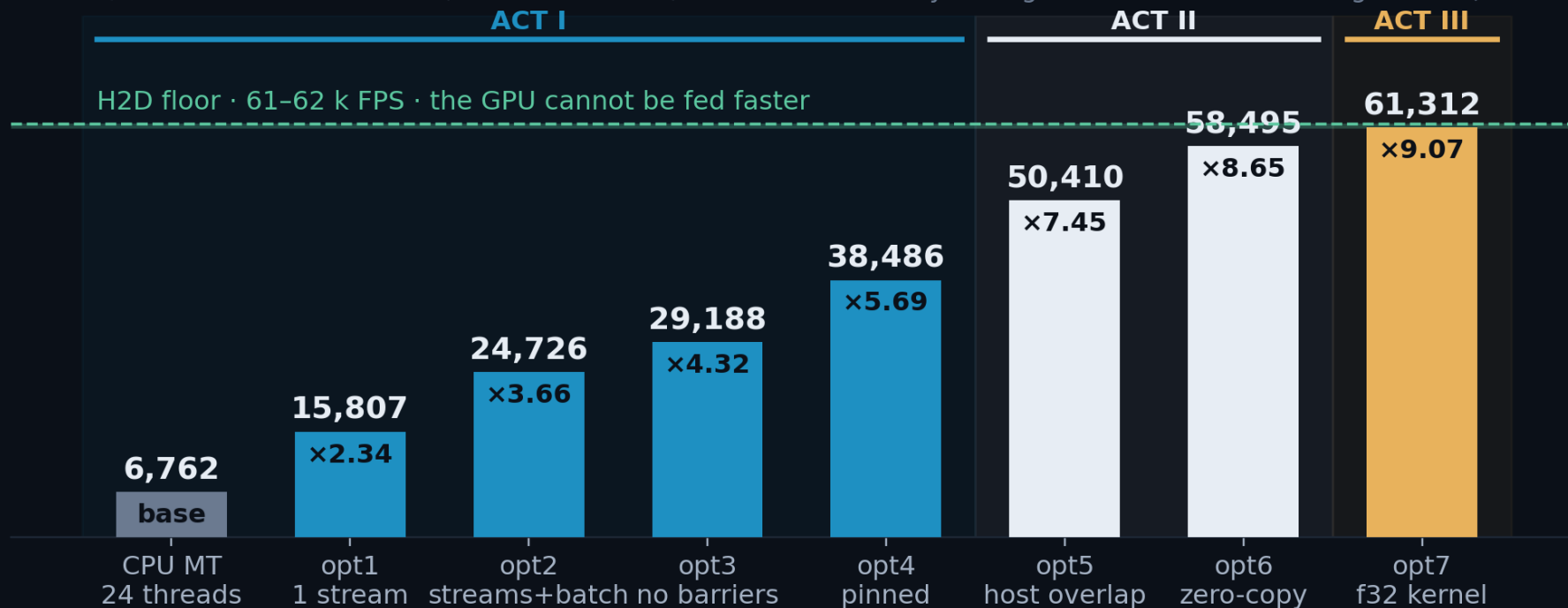
opt6 · everything after this is the ladder seen whole

COMING UP

- 25– Results, both cluster sizes
- 26
- 27 Where the time went
- 28– What the numbers survived
- 29

×9.1 at 3×3, sitting on the H2D floor

frames / second · 3×3 clusters, 100 000 frames, warm run · every bar against the best CPU configuration (24 threads)

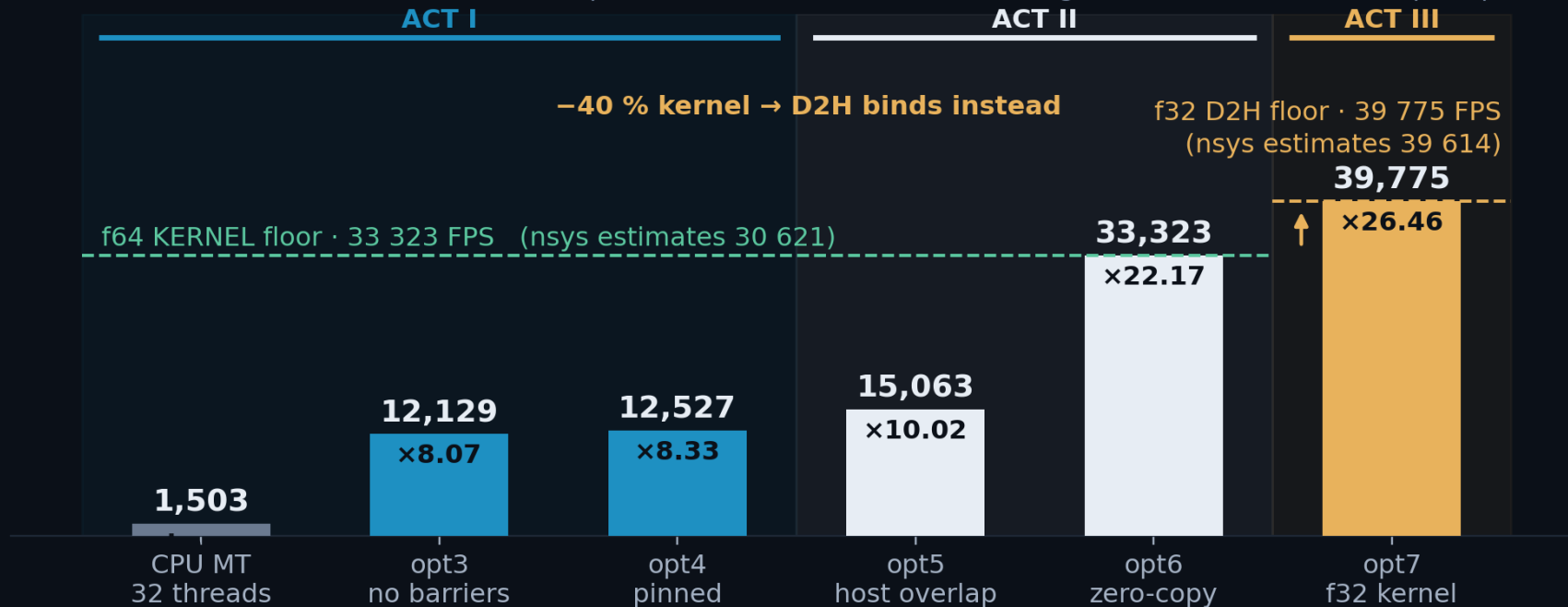


×9.1 over 24 CPU threads, 14.8 s → 1.63 s for 100 000 frames, and **at the H2D floor**.

Every step is **monotonic**, and correctness held constant throughout: **exact on the f64 pedestal, 6 clusters in 23 M** on the shipped f32.

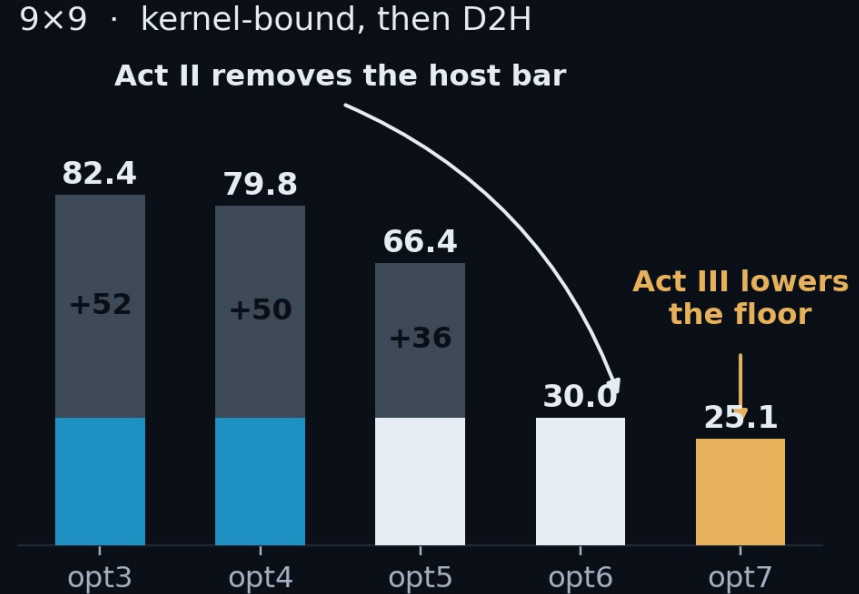
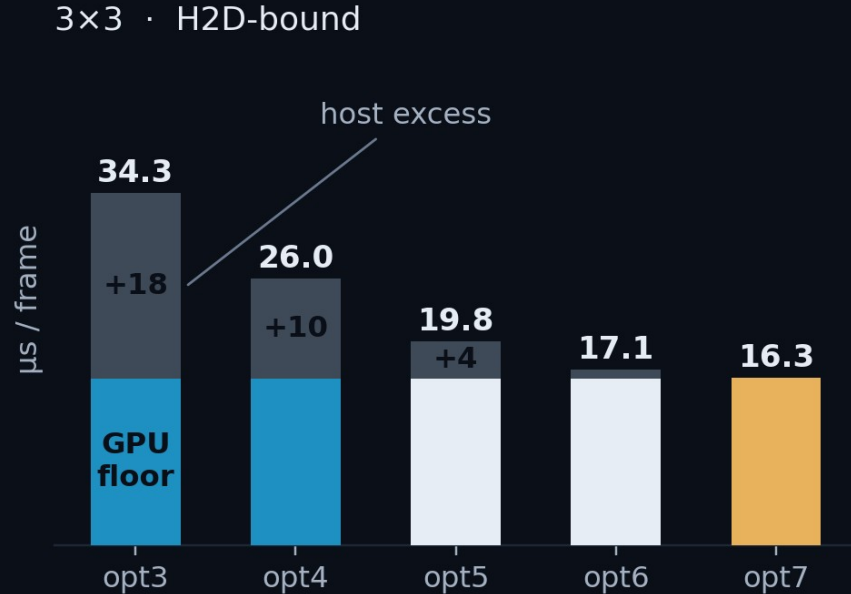
×26.5 at 9×9, and opt7 hands the floor to D2H

frames / second · 9×9, 20 000 frames, cap 1700 (lossless) · best CPU configuration (32 threads) · opt1/opt2 are 3×3 only



×26.5 over 32 CPU threads; the kernel is the tallest bar for the whole f64 arm, and opt7's –40 % drops it **below D2H**.

The host bar dies first, then the floor itself drops



Acts I and II never touch the arithmetic. The GPU floor is a flat 16.2 µs at 3×3 / 30.0 µs at 9×9; what collapses is everything stacked on it.

Act III is the only step that lowers the floor itself, and it could not have been seen until the stack above it was gone.

Coloured = the GPU floor for that act's build. Grey = everything the host adds on top: +50 µs at opt3, nothing at opt6.

Materialising clusters costs 2.6 M page faults

- Every run that keeps its results **allocates a fresh heap and touches it once**. Linux only finds physical memory on **first touch**, so that pass is where the OS works — **inside the timer**.

VIRTUAL · the ClusterVector malloc just handed you



PHYSICAL · 4 kB frames the kernel actually owns
 — already mapped — first touch → fault

MINOR FAULT · in the kernel

- 1 traps into the kernel
- 2 finds a free frame
- 3 zeroes all 4 kB of it
- 4 maps it; the write proceeds

no disk: that would be a MAJOR fault

THE PROTOCOL · BRACKET EVERY TIMED CELL

```
# bracket every timed cell:
mf0 = resource.getrusage(resource.RUSAGE_SELF).ru_minflt
... t = time.perf_counter() - t0 ...
print(f'minor faults: {mf1-mf0:,}') # quote the run where this plateaus
```

Re-run until minor faults plateau (< 200 k), and quote that run.

ONE 100 000-FRAME PASS · 3×3

PAGES FAULTED IN

~2.6 M

COST INSIDE THE TIMER

up to 4 s

CLUSTERS MATERIALISED

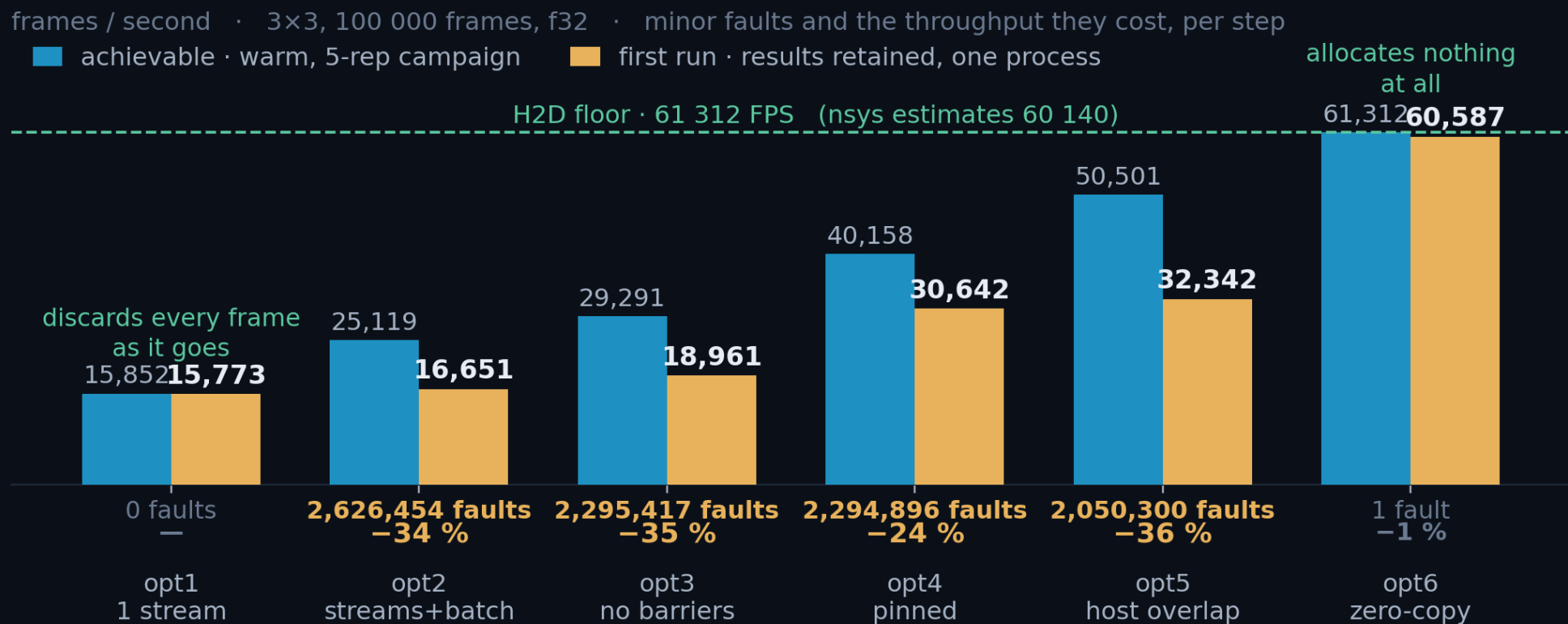
~10 GB

COST PER FAULT, FITTED

0.68 μs

Zero major faults all campaign. Copying faster cannot avoid this; not allocating can. That is opt6.

A first run loses a third of its throughput to page faults



Everything that materialises clusters loses a third of its throughput on the first run: +7 to +20 μ s per frame.

Only the two ends escape, for opposite reasons. opt1 discards each frame; opt6 never grows a heap, and reaches **98 % of its peak on a cold process.**

VALIDATION · DOES IT FIND THE SAME PHOTONS

Every number so far assumed the answers are identical

Whether the CUDA finder returns the same clusters
as the CPU.

ESTABLISHED

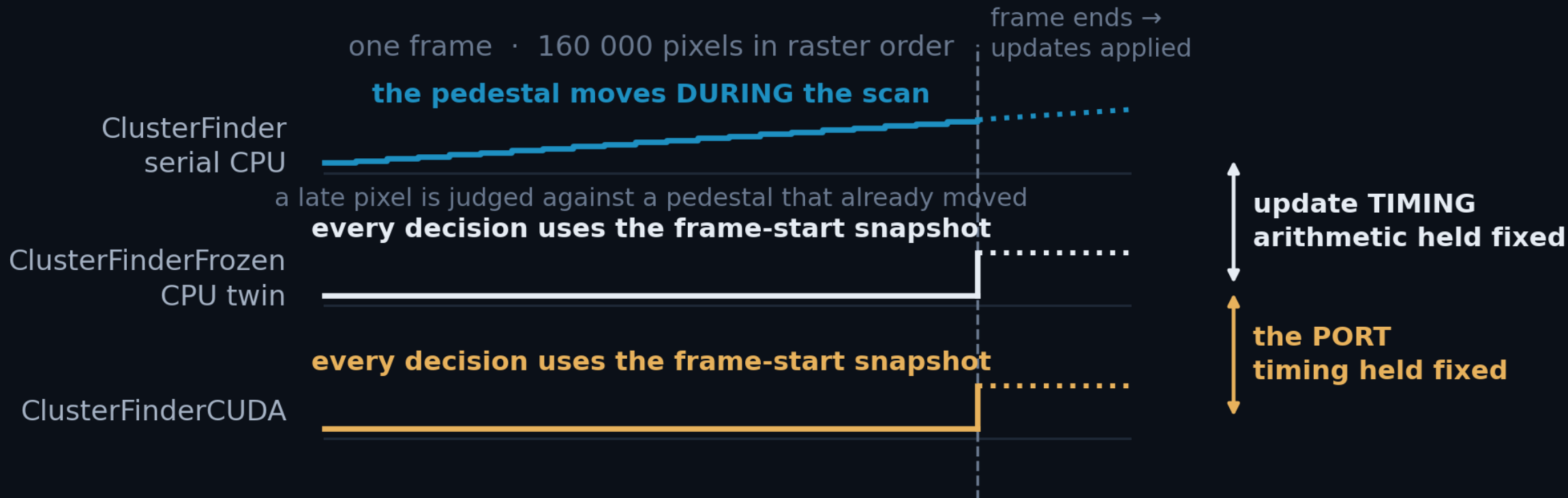
×9.1 and ×26.5

on the hardware floor at both cluster sizes, if the physics holds

COMING UP

- 30– The fair comparison
- 31
- 32– The residual, dissected
- 33
- 34– For users
- 35
- 36 What is next

CPU and CUDA update the pedestal at different moments



The CPU finder updates the pedestal **as the raster reaches each pixel**. 160 000 CUDA threads read it at once, so the update is **applied at the frame boundary**.

A straight CPU↔CUDA comparison therefore moves **two** things at once. **ClusterFinderFrozen** moves only the update: same arithmetic, same gates, same scan.

CUDA and its CPU twin agree exactly: 0 in 23 million

- **ClusterFinderFrozen** makes byte-for-byte the same decisions as ClusterFinder and differs in exactly one thing: **when** the pedestal is updated. Frozen per frame, pushed at frame end. That is the CUDA model, so comparing against it isolates everything else the port changes.

COMPARISON	THE ONE THING THAT DIFFERS	A-ONLY / B-ONLY	% OF CLUSTERS
serial CPU vs frozen CPU	update timing alone: a CPU-only effect	8 / 11	0.000082 %
serial CPU vs CUDA [f64 ped]	the same thing, and nothing else	8 / 11	0.000082 %
frozen CPU vs CUDA [f64 ped]	nothing	0 / 0	0 %
frozen CPU vs CUDA [f32 ped]	the float32 pedestal EMA drifting: see next slide	0 / 6	0.000026 %

The ringed pair is the port on its own. At f64 the two finders agree on all 23 244 605 clusters. Every difference in the rows above it is a CPU-vs-CPU effect; the six at f32 are the next slide.

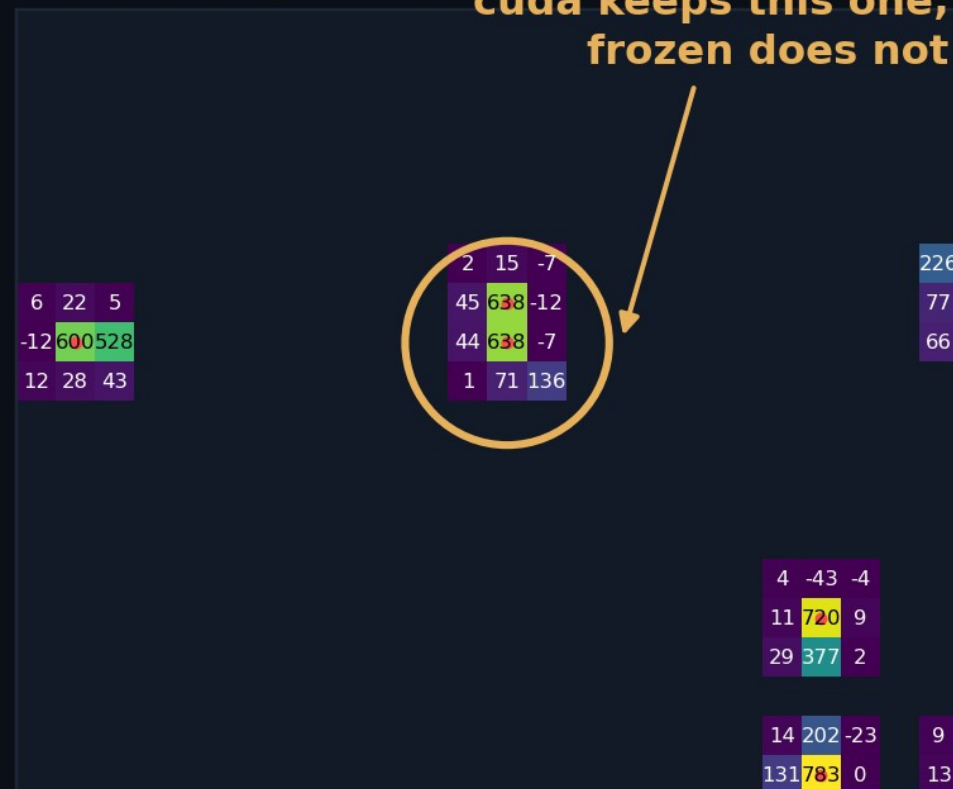
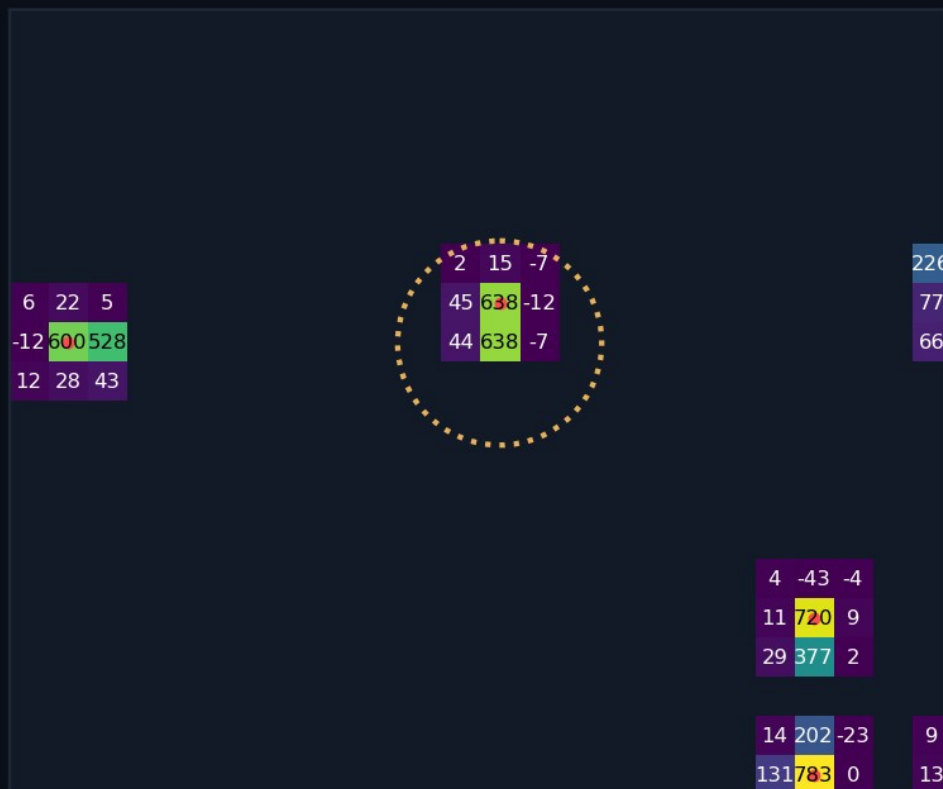
The whole disagreement is one duplicate centre

frozen — 2,270 clusters in this frame

cuda — 2,271 clusters in this frame

cuda keeps this one;
frozen does not

frame 147
zoom on (202, 8)



Only each finder's **own** 3×3 footprints are drawn, so the panels differ **exactly** where the finders do.

cuda's patch is **one row taller**: a second centre directly below the one both found. **The charge is already counted**: a duplicate, not a new photon.

float32 cannot tell these two pixels apart

```
frame 147    centre (x=202, y=8)    3x3 window
raw window (ADU)    pedestal-subtracted, 1 decimal
[[4646 5282 4703]    [[ 45.3 638.4 -12.1]    frozen and cuda
[4857 5318 4950]    [ 43.7 638.4 -7.5]    print the SAME
[4763 4640 4858]]    [ 1.1 70.7 136.4]]    window
```

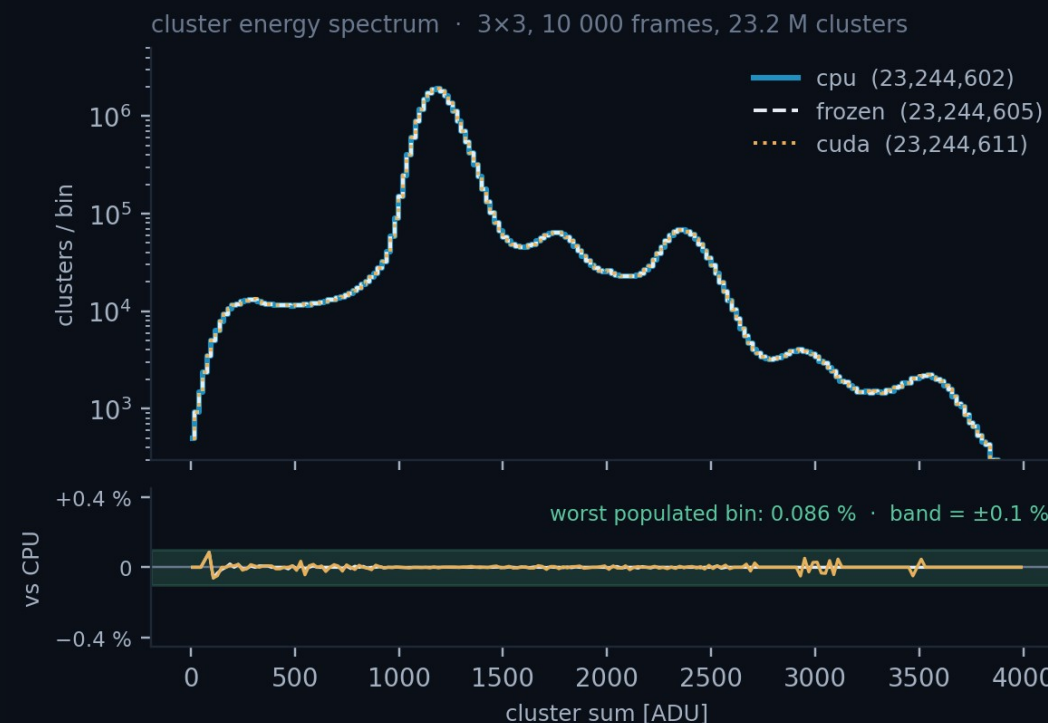
the two contenders, at full precision:

	rival (dy=-1)	centre
frozen [f64 ped]	638.383019956	638.382773664
cuda [f32 ped]	638.382812500	638.382812500

```
gate: accept if centre >= max(window)
frozen 638.382773664 >= 638.383019956 -> reject
cuda 638.382812500 >= 638.382812500 -> accept
```

Under the f64 pedestal the two pixels are **0.000246 ADU** apart; one float32 ULP here is **0.000488**. **Half a ULP** — in float32, the same number.

At f64 the residual is zero.



The fast path in eight lines

THE RECOMMENDED PATTERN

```
from aare import File, ClusterFinderCUDA
cf = ClusterFinderCUDA(image_size=(400, 400), cluster_size=(3, 3),
                        n_sigma=5, n_streams=4,
                        max_clusters_per_frame=3000)
for _ in range(1000):
    cf.push_pedestal_frame(pd.read_frame())
data = f.read_n(100_000)
cf.register_input_buffer(data)
for s in range(0, len(data), 2000):
    clusters = cf.find_clusters_batched(data[s:s+2000], first_frame=s)
cf.unregister_input_buffer()
```

1. train the pedestal
2. one contiguous array
3. pin it once
4. batch through it
5. release the pages

ZERO-COPY · SAME PATTERN, STEP 4 SWAPPED

```
for v in cf.find_cluster_views_batched_iter(data, 2000):
    hist.fill(v.sums())
```

4, zero-copy
consume inside the loop:
the view dies with its chunk

- **3 · register_input_buffer()** — opt4 in one call. Pin **once**, outside the loop.
- **4 · find_clusters_batched()** — one ClusterVector per frame, with opt5's overlap built in.

Zero-copy: ×1.16 at 3×3, ×2.21 at 9×9. The views expose every cluster; they only withhold **ownership** past the chunk.

Reduce as you go and you never allocate. Hold a view past its chunk and you stall the pipeline.

Five knobs, and the one that silently truncates

The single most common mistake: leaving **max_clusters_per_frame** too low. It does not error; it truncates, and every frame quietly returns the same count.

PARAMETER	WHAT IT DOES	GUIDANCE
<code>n_streams</code>	Upper bound on frames in flight.	4 at both cluster sizes. 8 buys no kernel concurrency at 9×9.
<code>max_clusters_per_frame</code>	Fixed size of the per-frame D2H transfer.	Must exceed the real maximum or clusters are silently dropped. At 9×9 the maximum is 1 633, and a cap of 1 700 already makes D2H the bottleneck.
<code>batch_size</code>	Frames per <code>find_clusters_batched</code> call.	2 000 amortises launch overhead without a large pinned footprint.
<code>cluster_size</code>	Compile-time stencil geometry.	3×3 and 9×9 are registered. 9×9 moves the bottleneck off H2D.
<code>register_input_buffer</code>	Page-locks the host array for DMA.	Always, if the data is already in RAM.

The bottleneck has walked from the host, to the GPU, to PCIe

Every number in this deck is measured on one machine, one dataset, one git revision — and the annexes carry the reconciliation for each of them.

DONE

×9.1 at 3×3, ×26.5 at 9×9

16.3 and 25.1 μs/frame end to end, both on their hardware floor. At 3×3 that is 45× MÖNCH03's standard frame rate.

DONE

FP32 pedestal, safely

—40.5 % kernel time, and correct: the variance accumulates on a frozen per-pixel offset. 6 duplicate clusters in 23 million.

NEXT

3×3: transfer granularity

H2D sustains 16.31 μs against 13.15 uncontended — 2 000 separate 320 kB descriptors, plus H2D↔D2H contention.

NEXT

9×9: the result path, not the kernel

At a lossless cap D2H already binds: 25.24 μs against a 23.94 μs kernel. More kernel work buys nothing.

Annexes

EVERYTHING SO FAR

35 slides

the arc is finished; what follows answers questions

COMING UP

- A1** Every engine number, reconciled
- A2** The three rejected routes
- A3** opt5 · the overlap code
- A4** The fault model, tested
- A5** The variance rewrite in full
- A6** The three benchmark artefacts
- A7** Which test causes the CPU/CPU gap

Uncontended, or as the pipeline runs it

• Every engine time in this deck is tagged **[build · s1|s4]**. **s1** is one stream with nothing else running: what an engine does **on its own**, which is the right number for a capability claim and for the headroom that remains. **s4** is the shipped four-stream pipeline: each engine's **busy time per frame**, the union of its intervals, the only number that can set a floor.

3×3 · μS/FRAME	S1 F64	S1 F32	S4 F64	S4 F32
H2D	13.14	13.15	16.17	16.63
kernel	14.72	4.32	15.17	5.53
D2H	5.31	5.27	7.69	7.57
engine max [s4]	—	—	16.17	16.63
FLOOR = lower of max, sustained	—	—	16.17	16.31

9×9 · CAP 1700	S1 F64	S1 F32	S4 F64	S4 F32
H2D	13.20	13.22	20.77	20.54
kernel	39.86	23.70	32.66	23.94
D2H	21.97	21.95	25.25	25.24
engine max [s4]	—	—	32.66	25.24
FLOOR = lower of max, sustained	—	—	30.01	25.14

Two traps. (1) s4 is engine *occupancy*, not duration — the 9×9 kernel row **falls** under load. (2) the **floor is not the engine max**: the max is profiled and runs 2-8 % high.

The D2H shift is an s4 phenomenon: at s1 the kernel binds in both arms. Which engine binds must be read from the s4 columns.

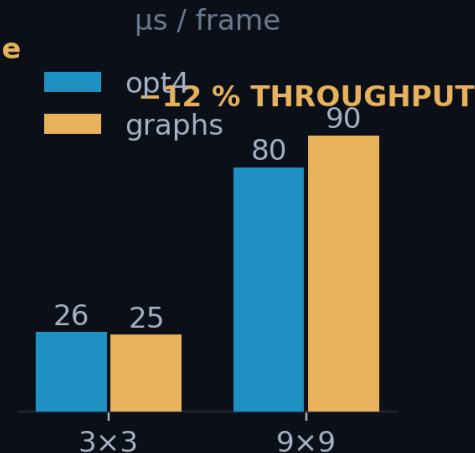
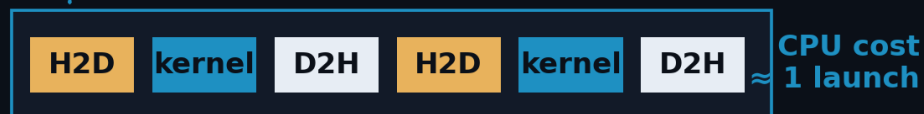
CUDA Graphs, a sound idea that the next act overtook

- Every cudaMemcpyAsync / kernel launch costs the **CPU** a few microseconds of driver work, per frame and per operation. After opt4 that looked like the budget.
- A **CUDA Graph** captures the whole dependency DAG once. Replaying it is a **single** cudaGraphLaunch: the driver

WITHOUT GRAPHS · one driver call per operation, every frame



WITH GRAPHS · record once, replay with one launch



ClusterFinderCUDA_graph.hpp

```
// record once, at setup
cudaStreamBeginCapture(sc.stream, ...);
    submit_h2d_kernel_d2h(sc);
cudaStreamEndCapture(sc.stream, &sc.graph);
cudaGraphInstantiate(&sc.graphExec, ...);

// per batch: one call
cudaGraphLaunch(sc.graphExec, sc.stream);
```

REJECTED

3x3: 39 752 FPS, inside noise of opt4.

9x9: 11 072 FPS, 12 % slower.

The 3x3 edge was never established: the instrumentation tax (2.8 μs) is larger than the gap (0.8 μs). Detail in the notes.

What a CUDA Graph actually saves, in microseconds

- The stream path issues **four runtime calls per frame** — memset, H2D, launch, D2H — and a graph replaces all four with **one** cudaGraphLaunch.

CALL	PER FRAME	HOST COST	μS/FRAME
cudaMemcpyAsync	2	1.98 μs	3.97
cudaLaunchKernel	1	2.13 μs	2.13
cudaMemsetAsync	1	1.57 μs	1.57

submission total

4

7.67

THE ARITHMETIC

7.67 us/frame x 3/4 = 5.75 us/frame eliminated, AS MEASURED (under nsys)

5.75 / 4 (see A5) ~ 1.4 us/frame eliminated, unprofiled estimate

~1.4 μs against a 16.17 μs floor = 8.7 %, real while the host is the critical path, and **worth nothing after opt5**, which hides host work under the GPU entirely.

ROUTE A · MEASURED VERDICT

3×3 VS OPT4

×1.03

9×9 VS OPT4

×0.88

VS OPT5 AT 9×9

36 % behind

Read from CUPTI_..._RUNTIME, the host table, so this is the one order-of-magnitude number in the deck, ±50 %. It is enough to show graphs cannot pay against a 16 μs floor, and not enough to quote to two figures.

The copy is allocation-bound, not bandwidth-bound

Copying faster does not help when the cost is the OS populating pages. The only winning move is not to allocate, which is exactly what opt6 does. `materialize_slot()` is deliberately single-threaded and carries a comment saying so, to stop the experiment being repeated.

B' One allocation per chunk

`collect_packed()`

Removes the per-frame `malloc` but keeps the copy, and the copy is ~80% of the cost. Worse, the replacement allocation is 1.17 GB, far above glibc's `mmap` threshold, so it is `mmap'd` and `munmap'd` every chunk: 606 566 faults, ~21 μ s/frame.

69.3 μ s · deleted from the API

B'' Parallel materialisation

`8-thread copy pool`

Each worker gets its own glibc arena, which destroys the cross-run heap reuse that makes the single-threaded path cheap. Faults went 9 700 → 2 270 000; `MALLOC_ARENA_MAX=1` collapsed them back to 138 k, which is the proof.

+6% at best, –33% when results are freed promptly

The overlap, in six lines, and why you never write them

THE WHOLE OF OPT5

```
tok = cf.submit_batch(data[a0:b0], first_frame=a0)
for a, b in bounds[1:]:
    nxt = cf.submit_batch(data[a:b], first_frame=a)    # GPU starts N+1 ...
    results.extend(cf.collect(tok))                    # ... host unpacks N
    tok = nxt
results.extend(cf.collect(tok))                        # drain the last one
```

- **You do not write this.** It is inside `find_clusters_batched()`, so `opt5` arrived as a **speedup, not an API change**.
- `submit_batch()` and `collect()` stay public for streaming from a detector, or interleaving other work between chunks.

Two chunks in flight is enough. A third adds pinned memory and no overlap: the host is already busy for the whole time the GPU is.

The saving is $\min(\text{GPU}, \text{host})$ per chunk, so `opt5` pays most when the two terms are comparable, $\times 1.31$ at 3×3 , and least when one dominates, $\times 1.20$ at 9×9 , where the host term is roughly twice the GPU term. That gap is the diagnosis that motivates `opt6` (report §8.2).

CHUNK SIZING · THE TWO CONSTRAINTS

MULTIPLE OF

`n_streams`

CAPPED AT

`MAX_SLOT_BYTES`

A multiple of `n_streams`, because the device pedestal is per-stream: an uneven chunk leaves the four pedestals at different ages and the finder stops being reproducible.

Capped so the two pinned slots stay bounded — 544.5 kB per frame at 9×9 , cap 1 700. `chunk_size_for(n)` applies both rules.

The rewrite in full, and which pixels the error reached

- Freeze $\mathbf{X_0 = round(mean)}$ once, at the end of pedestal training, and never move it.
- Accumulate the **centred** $Y = X - X_0$; report $\mathbf{X_0 + sum/n}$. Both operands are now **O(rms)-sized**.

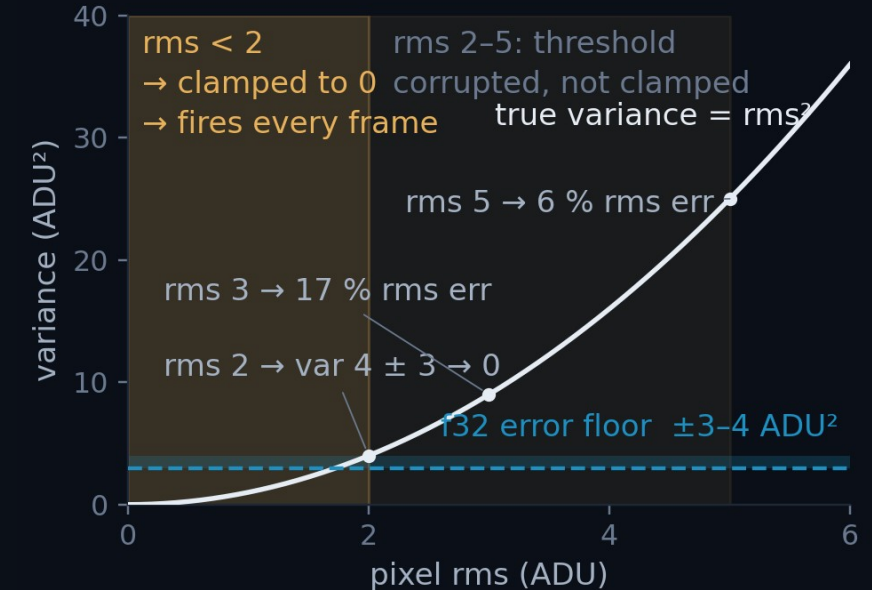
clusterfinder_kernel.cuh

```
// before: both terms ~2.17e7, answer ~2000
var = sum2/n - mean*mean;
```

```
// after: centred on a frozen per-pixel offset X0
DEVICE_PED_TYPE resid = mean - d_pd_off[i];    // ~0(1)
DEVICE_PED_TYPE var_px = sum2[i]/n - resid*resid; // no cancellation
```

Result: the 100 % FP32 build matches the FP64 build to 3×10^{-7} , 70 clusters out of 233 million.

X_0 must never be updated — the accumulators are defined relative to it. Welford's is the other correct answer; this one is two lines and costs nothing.



Which pixels the ± 3 ADU² floor actually reaches: the quiet ones, whose true variance is smallest.

Three ways a GPU benchmark lies

First-touch page faults

Each run materialises ~10 GB of clusters. The first pass faults in ~2.6 M pages at 0.7 μ s each, up to 4 s of pure OS work inside the timer.

Fix: re-run until getrusage() minor faults plateau (< 200 k).

CUDA-event kernel timing

avg_kernel_time_ms() measures elapsed time on a stream, including waiting for other streams. Under 8-stream load it over-reads by up to 3.5 $\times$.

Fix: Nsight Systems per-instance times; 1 stream for exclusive numbers.

The profiler itself

Under nsys, wall time per frame inflates ~4 $\times$ from API tracing.

Fix: GPU op times from nsys, wall times from unprofiled runs.

THE FAULT PROTOCOL · BRACKET EVERY TIMED CELL

```
# bracket every timed cell:
mf0 = resource.getrusage(resource.RUSAGE_SELF).ru_minflt
... t = time.perf_counter() - t0 ...
print(f'minor faults: {mf1-mf0:,}') # quote the run where this plateaus
```

Validated: **wall = steady-state + faults $\times$ 0.68 μ s** reproduced a 6.110 s run to within **1 ms**. Kernel time stayed constant throughout; the GPU was never the variable.

First-touch page faults: two sources, one counter

- A page exists in the address space but has no physical frame yet. First touch makes the kernel find one, **zero it** (mandatory), and map it. No disk: `ru_majflt` stays 0 all campaign. At 4 kB/page, **1 GB = 262 144 faults**.

	(A) RESULT HEAP	(B) PINNED D2H SLOTS
allocator	<code>mmap</code> → <code>mmap</code> , one <code>ClusterVector</code> per frame	<code>cudaMallocHost</code> , in <code>submit_batch</code>
cost / page	0.7 µs	1.0 µs : same fault + pin + DMA map
recurs?	yes , every alloc/free cycle above the <code>mmap</code> threshold	no : once per buffer, for its lifetime
removed by	collect_view() : it allocates nothing	nothing; <code>reserve_output_slots()</code> only moves it out of the timer

The two are exactly additive. Reserving subtracts precisely the pre-pin count from run 0 and changes nothing else.

NOT A CORRELATION, AN IDENTITY

3x3: 572 292 - 455 129 = 117 163 vs 117 192 pre-pin
9x9: 2 759 037 - 2 278 567 = 480 470 vs 480 474
closed form: 2 slots x 2000 x 120 004 B / 4 kB = 117 191

At 9×9 the heap never plateaus. ~9.3 GB per pass exceeds glibc's `mmap` threshold, so it is re-faulted every pass: **~10 µs/frame, permanently**.

CUDA events measure the stream, not the kernel

- `avg_kernel_time_ms()` brackets the launch with **cudaEventRecord on the kernel's own stream**, so it returns time on **that stream's timeline** — including time queued behind other streams.
- Honest at 1 stream, inflated up to **3.5x** at 8. The tell: `*PCIe + overhead* = wall/N - kernel_ms` **goes negative**.
- It is not free either: **~3.6 µs/frame**, 10-15 % of throughput at 3x3.

ClusterFinderCUDA.hpp

```
if (m_time_kernels)                // OFF by default, all 3 finders
    cudaEventRecord(start[slot][i], sc.stream);
find_clusters_in_single_frame<<<grid, block, shmem, sc.stream>>>(...);
if (m_time_kernels)
    cudaEventRecord(stop[slot][i], sc.stream); // <- queue-wait lands here
```

The flag exists for **comparability**, not preference: with events on for one finder and off for another, the step between them absorbs the tax.

9x9 KERNEL · F64 · NSYS

S1 · DURATION

39.86 µs

S4 · OCCUPANCY

32.66 µs

OVERLAP FACTOR

1.32x

Same kernel. The s4 column is the union of kernel intervals per frame, not how long one kernel takes. Quote s1 for duration; s4 feeds the floor, subject to the sustained-rate rule on slide 21. Full grid: A1.

Where nsys is sound, and where it is not

- Tracing a **host** call runs a callback on entry and exit, **inside the interval being measured**. GPU work is stamped by the hardware and read back **afterwards**, so nothing is injected into the execution path.

MEASUREMENT	SQLITE TABLE	9×9 [F64 · S1 · CAP 1500]	VERDICT
cudaLaunchKernel: the host call	CUPTI_..._RUNTIME	1.85 µs	inflated ~4×
the kernel executing	CUPTI_..._KERNEL	39.93 µs	sound to ~2 %
cudaMemcpyAsync: the host call	CUPTI_..._RUNTIME	1.65 µs	inflated ~4×
the H2D / D2H transfer	CUPTI_..._MEMCPY	13.25 / 19.44 µs	sound to ~2 %

Same cudaMemcpyAsync, two numbers: 1.65 µs to ask for the copy, 13.25 µs for the copy to happen. A profiler that must be present to measure, distorts; one that reads a record later does not.

Every engine number in this deck comes from GPU-side timestamps. Proof they are sound: opt7 sustains 25.14 µs unprofiled against a 25.24 µs profiled estimate, 0.4 % apart.

There is a third test

- Slide 4 showed the decision with **two** tests, which is the right simplification for the arc. The finder has **three**, and the missing one is what the next slide is about.

ClusterFinder.hpp:104-142 · all three branches

```
v = frame[i] - ped_mean[i];  rms = ped_rms[i]
if (v < -nSigma*rms)         -> skip, no update
m    = max over the 3x3 window
total = sum over the 3x3 window
if (m > nSigma*rms)           // TEST 1
    if (v == m) -> emit cluster // local-max gate
    else       -> shadow, no update
else if (total > c3*nSigma*rms) // TEST 3
    if (v == m) -> emit cluster
    else       -> update pedestal
```

WHAT EACH TEST ASKS

TEST 1 · NSIGMA · RMS

is any ONE pixel bright?

TEST 3 · C3 · NSIGMA · RMS

is the WINDOW bright?

LOCAL-MAX GATE

is this pixel the peak?

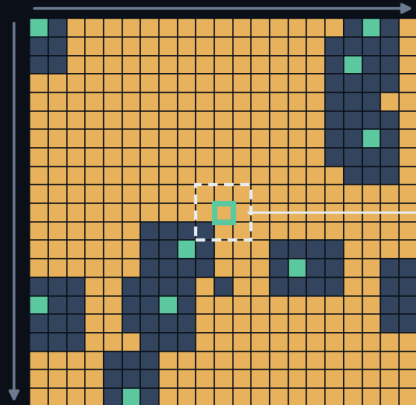
Test 1 reads the MAX of the window; Test 3 reads its SUM. That one difference is the whole of the next slide.

$c3 = \sqrt{3 \times 3} = 3$ falls out of variance addition. Variances add, standard deviations do not: nine pixels of noise σ give **$\text{Var}(\Sigma v) = 9\sigma^2$** , so the sum's noise is **3σ** . **$c3 \cdot nSigma \cdot rms$** is therefore the*same 5σ criterion as Test 1*, asked of the window instead of the pixel.

Test3 makes the clusters; Test1 only moves the pedestal

- **push_fast** touches only the pixel's **own** accumulators and never reads the stencil, so the update is **order-independent**: same set of updated pixels, bit-identical pedestal. Only a differing **decision** can diverge.

the finished frame · 21 × 21 around (125, 245)
raster order



the pixel they disagree on

- stored: this pixel IS the window max, and it clears 5σ
- shadow: the window max clears 5σ , but this pixel is not it
- pedestal sample: nothing in the window clears 5σ
- the disputed pixel — serial sampled here, frozen stored

the moment (125, 245) was tested · only the 4 already-scanned neighbours differ

31.669	14.420	33.763
31.637	14.405	33.730
41.680	56.473	30.007
41.638		
15.214	40.217	21.979

frozen above serial · amber ring = pushed pedestal

serial $\Sigma = 256.489 < 256.511 \rightarrow$ **pedestal sample**

frozen $\Sigma = 256.581 > 256.511 \rightarrow$ **CLUSTER**

max = 56.473 in both $\rightarrow$ **Test1 never moved**

Measured two ways. **Instrumented**: of the 19 disagreeing clusters, the **11 that only frozen finds are all Test 3**; the 8 only serial finds are all the local-max gate. **Ablated**: compile Test 3 out and the 11 go to **zero**.